MIXED PRECISION TRAINING

Sharan Narang*, **Gregory Diamos, Erich Elsen**†
Baidu Research
{sharan, gdiamos}@baidu.com

Paulius Micikevicius*, **Jonah Alben, David Garcia, Boris Ginsburg, Michael Houston, Oleksii Kuchaiev, Ganesh Venkatesh, Hao Wu**
NVIDIA
{pauliusm, alben, dagarcia, bginsburg, mhouston, okuchaiev, gavenkatesh, skyw}@nvidia.com

arXiv:1710.03740v3 [cs.AI] 15 Feb 2018

ABSTRACT

Increasing the size of a neural network typically improves accuracy but also increases the memory and compute requirements for training the model. We introduce methodology for training deep neural networks using half-precision floating point numbers, without losing model accuracy or having to modify hyper-parameters. This nearly halves memory requirements and, on recent GPUs, speeds up arithmetic. Weights, activations, and gradients are stored in IEEE half-precision format. Since this format has a narrower range than single-precision we propose three techniques for preventing the loss of critical information. Firstly, we recommend maintaining a single-precision copy of weights that accumulates the gradients after each optimizer step (this copy is rounded to half-precision for the forward- and back-propagation). Secondly, we propose loss-scaling to preserve gradient values with small magnitudes. Thirdly, we use half-precision arithmetic that accumulates into single-precision outputs, which are converted to half-precision before storing to memory. We demonstrate that the proposed methodology works across a wide variety of tasks and modern large scale (exceeding 100 million parameters) model architectures, trained on large datasets.

1 INTRODUCTION

Deep Learning has enabled progress in many different applications, ranging from image recognition (He et al., 2016a) to language modeling (Jozefowicz et al., 2016) to machine translation (Wu et al., 2016) and speech recognition (Amodei et al., 2016). Two trends have been critical to these results - increasingly large training data sets and increasingly complex models. For example, the neural network used in Hannun et al. (2014) had 11 million parameters which grew to approximately 67 million for bidirectional RNNs and further to 116 million for the latest forward only Gated Recurrent Unit (GRU) models in Amodei et al. (2016).

Larger models usually require more compute and memory resources to train. These requirements can be lowered by using reduced precision representation and arithmetic. Performance (speed) of any program, including neural network training and inference, is limited by one of three factors: arithmetic bandwidth, memory bandwidth, or latency. Reduced precision addresses two of these limiters. Memory bandwidth pressure is lowered by using fewer bits to to store the same number of values. Arithmetic time can also be lowered on processors that offer higher throughput for reduced precision math. For example, half-precision math throughput in recent GPUs is 2× to 8× higher than for single-precision. In addition to speed improvements, reduced precision formats also reduce the amount of memory required for training.

Modern deep learning training systems use single-precision (FP32) format. In this paper, we address the training with reduced precision while maintaining model accuracy. Specifically, we train vari-

*Equal contribution
†Now at Google Brain eriche@google.com

MIXED PRECISION TRAINING

Sharan Narang*、**Gregory Diamos**、**Erich Elsen**†百度研究院{sharan, gdiamos}@baidu.com

Paulius Micikevicius*、**Jonah Alben**、**David Garcia**、**Boris Ginsburg**、**Michael Houston**、**Oleksii Kuchaiev**、**Ganesh Venkatesh**、**Hao Wu**NVIDIA

{pauliusm,alben, dagarcia, bginsburg, mhouston, okuchaiev, gavenkatesh, skyw}@nvidia.com

arXiv:1710.03740v3 [cs.AI] 15 Feb 2018

ABSTRACT

通常情况下，增加神经网络的规模能够提升精度，但同时也会提高训练模型所需的内存和计算资源。我们提出了一种利用半精度浮点数来训练深度神经网络的方法，无需牺牲模型精度，也无需修改超参数。该方法几乎可将内存需求降低一半，同时在现代GPU上还能加快运算速度。权重、激活值以及梯度均以IEEE半精度格式存储。由于该格式的数值范围小于单精度格式，我们设计了三种技术以避免关键信息的丢失。首先，我们建议在每次优化器迭代后保留一份单精度的权重副本，用于累积梯度（该副本在正向传播和反向传播过程中会被转换为半精度）。其次，我们采用损失缩放技术来保留幅度较小的梯度值。最后，我们使用半精度运算并将其结果累积为单精度输出，然后再在存储到内存之前将其转换回半精度。实验表明，所提出的方法在多种任务以及现代大规模（参数量超过1亿）的模型架构中均能有效发挥作用，且这些模型都是在海量数据集上训练得到的。

1 引言

深度学习推动了众多不同应用领域的进步，涵盖从图像识别（He等人，2016a）到语言建模（Jozefowicz等人，2016），再到机器翻译（Wu等人，2016）以及语音识别任务（Amodei等人，2016）。实现这些成果的关键因素主要有两大趋势：日益庞大的训练数据集以及愈发复杂的模型结构。例如，Hannun等人（2014）所使用的神经网络拥有1100万个参数，而双向RNN的参数量增长至约6700万，到了Amodei等人（2016）研究的最新仅支持正向处理的门控循环单元（GRU）模型中，这一数字更是上升到了1.16亿。

更大的模型在训练时通常需要更多的计算能力和内存资源。通过采用精度降低的表示方式与运算方法，可以降低这些需求。包括神经网络训练和推理在内的任何程序的性能（速度）都受到三个因素中的某一方的限制：运算带宽、内存带宽或延迟。精度降低能够解决其中两个限制因素。通过使用更少的比特数来存储相同数量的数值，可以有效减轻内存带宽的压力。在那些针对低精度运算具备更高吞吐量的处理器上，运算时间也能得到缩短。例如，最新款GPU的半精度运算吞吐量比单精度高 2× 到 8× 倍。除了提升速度之外，精度降低的格式还能减少训练所需的内存容量。

Mo目前的深度学习训练系统大多采用单精度（FP32格式）进行运算。在本文中，我们针对这一问题进行了探讨。 s
the 在保持模型准确率的同时使用较低精度进行训练。具体而言，我们对var进行训练 i-

*同等贡献
†现任职于Google Brain，邮箱：eriche@google.com

ous neural networks using IEEE half-precision format (FP16). Since FP16 format has a narrower dynamic range than FP32, we introduce three techniques to prevent model accuracy loss: maintaining a master copy of weights in FP32, loss-scaling that minimizes gradient values becoming zeros, and FP16 arithmetic with accumulation in FP32. Using these techniques we demonstrate that a wide variety of network architectures and applications can be trained to match the accuracy FP32 training. Experimental results include convolutional and recurrent network architectures, trained for classification, regression, and generative tasks. Applications include image classification, image generation, object detection, language modeling, machine translation, and speech recognition. The proposed methodology requires no changes to models or training hyper-parameters.

2 Related Work

There have been a number of publications on training Convolutional Neural Networks (CNNs) with reduced precision. Courbariaux et al. (2015) proposed training with binary weights, all other tensors and arithmetic were in full precision. Hubara et al. (2016a) extended that work to also binarize the activations, but gradients were stored and computed in single precision. Hubara et al. (2016b) considered quantization of weights and activations to 2, 4 and 6 bits, gradients were real numbers. Rastegari et al. (2016) binarize all tensors, including the gradients. However, all of these approaches lead to non-trivial loss of accuracy when larger CNN models were trained for ILSVRC classification task (Russakovsky et al., 2015). Zhou et al. (2016) quantize weights, activations, and gradients to different bit counts to further improve result accuracy. This still incurs some accuracy loss and requires a search over bit width configurations per network, which can be impractical for larger models. Mishra et al. improve on the top-1 accuracy achieved by prior weight and activation quantizations by doubling or tripling the width of layers in popular CNNs. However, the gradients are still computed and stored in single precision, while quantized model accuracy is lower than that of the widened baseline. Gupta et al. (2015) demonstrate that 16 bit fixed point representation can be used to train CNNs on MNIST and CIFAR-10 datasets without accuracy loss. It is not clear how this approach would work on the larger CNNs trained on large datasets or whether it would work for Recurrent Neural Networks (RNNs).

There have also been several proposals to quantize RNN training. He et al. (2016c) train quantized variants of the GRU (Cho et al., 2014) and Long Short Term Memory (LSTM) (Hochreiter and Schmidhuber, 1997) cells to use fewer bits for weights and activations, albeit with a small loss in accuracy. It is not clear whether their results hold for larger networks needed for larger datasets Hubara et al. (2016b) propose another approach to quantize RNNs without altering their structure. Another approach to quantize RNNs is proposed in Ott et al. (2016). They evaluate binary, ternary and exponential quantization for weights in various different RNN models trained for language modelling and speech recognition. All of these approaches leave the gradients unmodified in single-precision and therefore the computation cost during back propagation is unchanged.

The techniques proposed in this paper are different from the above approaches in three aspects. First, all tensors and arithmetic for forward and backward passes use reduced precision, FP16 in our case. Second, no hyper-parameters (such as layer width) are adjusted. Lastly, models trained with these techniques do not incur accuracy loss when compared to single-precision baselines. We demonstrate that this technique works across a variety of applications using state-of-the-art models trained on large scale datasets.

3 Implementation

We introduce the key techniques for training with FP16 while still matching the model accuracy of FP32 training session: single-precision master weights and updates, loss-scaling, and accumulating FP16 products into FP32. Results of training with these techniques are presented in Section 4.

3.1 FP32 master copy of weights

In mixed precision training, weights, activations and gradients are stored as FP16. In order to match the accuracy of the FP32 networks, an FP32 master copy of weights is maintained and updated with the weight gradient during the optimizer step. In each iteration an FP16 copy of the master weights is

我们使用IEEE半精度格式（FP16格式）来构建神经网络。由于FP16格式的动态范围比FP32更窄，因此我们采用了三种技术来避免模型精度下降：以FP32格式保留权重的主副本、通过损失缩放来防止梯度值变为零，以及采用在FP32中进行累积的FP16运算方式。借助这些技术，我们证明了多种不同的网络架构和应用都可以被训练出与FP32格式训练相当的精度。实验结果涵盖了用于分类、回归和生成任务的卷积网络及循环网络架构；应用场景则包括图像分类、图像生成、目标检测、语言建模、机器翻译以及语音识别任务。所提出的方法无需对模型或训练超参数进行任何修改。

2 相关工作

There have been a number of publications on training Convolutional Neural Networks (CNNs) with reduced precision. Courbariaux et al. (2015) proposed training with binary weights, all other tensors and arithmetic were in full precision. Hubara et al. (2016a) extended that work to also binarize the activations, but gradients were stored and computed in single precision. Hubara et al. (2016b) considered quantization of weights and activations to 2, 4 and 6 bits, gradients were real numbers. Rastegari et al. (2016) binarize all tensors, including the gradients. However, all of these approaches lead to non-trivial loss of accuracy when larger CNN models were trained for ILSVRC classification task (Russakovsky et al., 2015). Zhou et al. (2016) quantize weights, activations, and gradients to different bit counts to further improve result accuracy. This still incurs some accuracy loss and requires a search over bit width configurations per network, which can be impractical for larger models. Mishra et al. improve on the top-1 accuracy achieved by prior weight and activation quantizations by doubling or tripling the width of layers in popular CNNs. However, the gradients are still computed and stored in single precision, while quantized model accuracy is lower than that of the widened baseline. Gupta et al. (2015) demonstrate that 16 bit fixed point representation can be used to train CNNs on MNIST and CIFAR-10 datasets without accuracy loss. It is not clear how this approach would work on the larger CNNs trained on large datasets or whether it would work for Recurrent Neural Networks (RNNs).

目前也有不少关于对RNN训练进行量化的方案被提出。He等人（2016c）训练了经过量化的GRU单元（Cho等人，2014年）以及长短期记忆（LSTM）单元（Hochreiter和Schmidhuber，1997年），旨在用更少的比特数来表示权重和激活值，尽管这会带来一定的精度损失。目前尚不清楚他们的研究结果是否适用于处理更大数据集时所需的更大规模网络。Hubara等人（2016b）提出了另一种无需改变RNN结构即可对其进行量化的方法。Ott等人（2016年）也提出了另一种量化RNN的方案，他们针对用于语言建模和语音识别训练的各种不同RNN模型，评估了二进制量化、三进制量化以及指数量化三种权重量化方式的效果。所有这些方法都不会修改单精度下的梯度，因此反向传播过程中的计算成本保持不变。

本文提出的技术在与上述方法的对比中存在三个不同之处。首先，前向与反向传播过程中的所有张量及运算均采用降低精度的FP16格式。其次，不会调整任何超参数（如层宽度）。最后，使用这些技术训练得到的模型在精度方面并不会低于单精度基准方案。我们通过基于大规模数据集训练的先进模型，证明了该技术在多种应用场景中均能有效发挥作用。

3 实现方式

本文介绍了在采用FP16格式进行训练的同时仍能保持与FP32格式训练相同模型精度的关键技术，包括单精度主权重及其更新机制、损失缩放技术，以及将FP16格式下的计算结果逐步累加为FP32格式的功能。第4节将展示运用这些技术所获得的训练结果。

3.1 FP32格式的主权重副本

在混合精度训练中，权重、激活值以及梯度会以FP16格式进行存储。为确保其精度能够与FP32网络相当，系统会维护一份FP32格式的权重主副本，并在优化器执行步骤时根据权重梯度对该主副本进行更新。在每次迭代中，都会基于这份主权重生成一份FP16格式的副本，

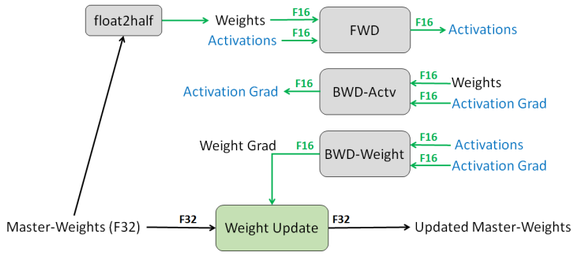


Figure 1: Mixed precision training iteration for a layer.

used in the forward and backward pass, halving the storage and bandwidth needed by FP32 training. Figure 1 illustrates this mixed precision training process.

While the need for FP32 master weights is not universal, there are two possible reasons why a number of networks require it. One explanation is that updates (weight gradients multiplied by the learning rate) become too small to be represented in FP16 - any value whose magnitude is smaller than 2^{-24} becomes zero in FP16. We can see in Figure 2b that approximately 5% of weight gradient values have exponents smaller than -24. These small valued gradients would become zero in the optimizer when multiplied with the learning rate and adversely affect the model accuracy. Using a single-precision copy for the updates allows us to overcome this problem and recover the accuracy.

Another explanation is that the ratio of the weight value to the weight update is very large. In this case, even though the weight update is representable in FP16, it could still become zero when addition operation right-shifts it to align the binary point with the weight. This can happen when the magnitude of a normalized weight value is at least 2048 times larger that of the weight update. Since FP16 has 10 bits of mantissa, the implicit bit must be right-shifted by 11 or more positions to potentially create a zero (in some cases rounding can recover the value). In cases where the ratio is larger than 2048, the implicit bit would be right-shifted by 12 or more positions. This will cause the weight update to become a zero which cannot be recovered. An even larger ratio will result in this effect for de-normalized numbers. Again, this effect can be counteracted by computing the update in FP32.

To illustrate the need for an FP32 master copy of weights, we use the Mandarin speech model (described in more detail in Section 4.3) trained on a dataset comprising of approximately 800 hours of speech data for 20 epochs. As shown in 2a, we match FP32 training results when updating an FP32 master copy of weights after FP16 forward and backward passes, while updating FP16 weights results in 80% relative accuracy loss.

Even though maintaining an additional copy of weights increases the memory requirements for the weights by 50% compared with single precision training, impact on overall memory usage is much smaller. For training memory consumption is dominated by activations, due to larger batch sizes and activations of each layer being saved for reuse in the back-propagation pass. Since activations are also stored in half-precision format, the overall memory consumption for training deep neural networks is roughly halved.

3.2 Loss scaling

FP16 exponent bias centers the range of normalized value exponents to $[-14, 15]$ while gradient values in practice tend to be dominated by small magnitudes (negative exponents). For example, consider Figure 3 showing the histogram of activation gradient values, collected across all layers during FP32 training of Multibox SSD detector network (Liu et al., 2015a). Note that much of the FP16 representable range was left unused, while many values were below the minimum representable range and became zeros. Scaling up the gradients will shift them to occupy more of the representable range and preserve values that are otherwise lost to zeros. This particular network diverges when gradients are not scaled, but scaling them by a factor of 8 (increasing the exponents by 3) is sufficient to match the accuracy achieved with FP32 training. This suggests that activation

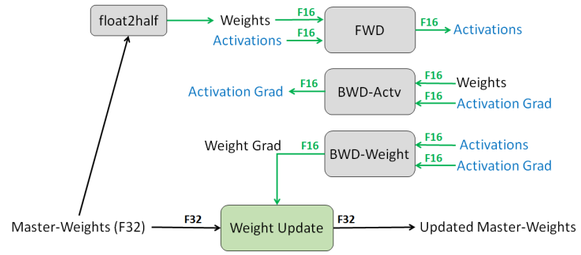


图1：某一层的混合精度训练迭代过程。

该FP16副本会被用于前向传播和反向传播过程，从而将FP32训练所需的存储空间和带宽需求降低一半。图1展示了这种混合精度训练的具体流程。

虽然并非所有场景都必需FP32格式的主权重，但仍有不少网络需要它，原因主要有两个。一种解释是，更新值（即权重梯度乘以学习率后的结果）在FP16格式中会变得过小而无法准确表示——凡是幅值小于 2^{-24} 的数值在FP16中都会被视为零。从图2b可以看出，大约5%的权重梯度数值的指数小于 -24。这类数值较小的梯度在与学习率相乘后会在优化器中被视为零，进而对模型精度产生负面影响。使用单精度副本进行更新即可解决这一问题，从而恢复模型的精度。

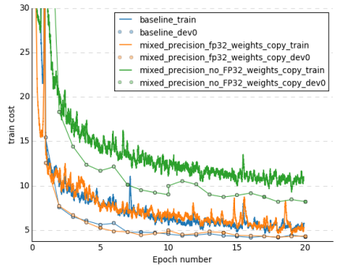
另一种解释是权重值与权重更新量的比值过大。在这种情况下，即便权重更新量可以用FP16格式表示，但在执行加法运算时为了使二进制小数点与权重对齐，该值仍可能被右移至零。当归一化后的权重值幅度至少是权重更新量的2048倍时，就会出现这种情况。由于FP16格式的尾数部分仅有10位，因此隐含位必须被右移11位或更多才能变为零（在某些情况下通过四舍五入可以恢复该数值）。若比值超过2048，隐含位则需要被右移12位或更多，这会导致权重更新量变为无法恢复的零。对于未归一化的数值，更大的比值同样会产生此类效应。同样地，通过在FP32格式下计算更新量，就可以避免这种问题。

为阐明为何需要权重值的FP32格式主副本，我们以普通话语音模型为例（其相关细节可见第4.3节）。该模型在包含约800小时语音数据的数据集上进行了20轮训练。如图2a所示，在完成FP16格式的前向与反向传播运算后，若更新权重值的FP32格式主副本，训练结果仍能保持一致；而直接更新FP16格式的权重值，则会导致相对精度损失达80%。

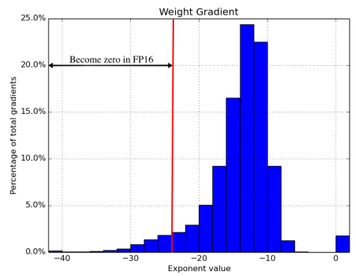
尽管保留额外的权重副本会使权重所需的内存占用比单精度训练高出50%，但对整体内存使用量的影响其实相当有限。在训练过程中，由于批量大小较大，且每层的激活值会被保存下来以便在反向传播阶段重复使用，因此激活值占据了主要的内存消耗。再加上激活值本身也是以半精度格式存储的，这使得深度神经网络训练时的整体内存消耗大致减少了一半。

3.2 损失缩放

FP16格式的指数偏置功能可将归一化值指数的范围锁定在 $[-14, 15]$，因为在实际应用中梯度值往往以较小数值（负指数）为主。例如，图3展示了在采用FP32格式对Multibox SSD检测器网络进行训练时（Liu等人2015年的研究），从所有层中收集到的激活梯度值的直方图。可以看出，FP16格式所能表示的范围中有很大部分未被利用，同时还有许多值低于其最小表示范围而变为零。通过放大梯度值，可以使它们占据更宽的表示范围，从而保留那些否则会因变为零而丢失的数值。若不进行梯度缩放，该网络会出现发散现象，但将梯度值放大8倍（即让指数增加3）就足以达到与FP32格式训练相同的准确率。这一现象表明激活函数



(a) Training and validation (dev0) curves for Mandarin speech recognition model



(b) Gradient histogram for Mandarin training run

Figure 2: Figure 2a shows the results of three experiemnts; baseline (FP32), pseudo FP16 with FP32 master copy, pseudo FP16 without FP32 master copy. Figure 2b shows the histogram for the exponents of weight gradients for Mandarin speech recognition training with FP32 weights. The gradients are sampled every 4,000 iterations during training for all the layers in the model.

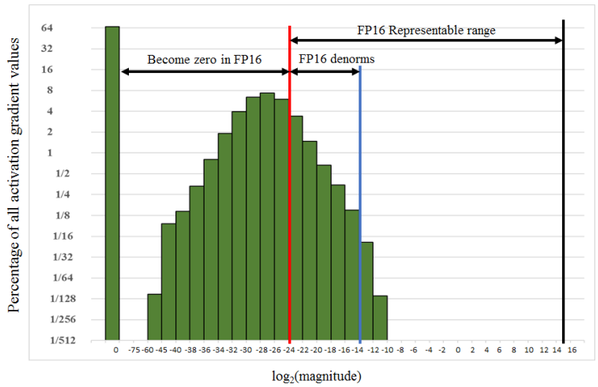
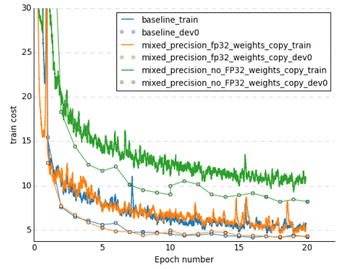


Figure 3: Histogram of activation gradient values during the training of Multibox SSD network. Note that the bins on the x-axis cover varying ranges and there's a separate bin for zeros. For example, 2% of the values are in the $[2^{-34}, 2^{-32})$ range, 2% of values are in the $[2^{-24}, 2^{-23})$ range, and 67% of values are zero.

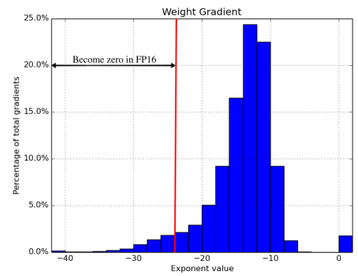
gradient values below 2^{-27} in magnitude were irrelevant to the training of this model, but values in the $[2^{-27}, 2^{-24})$ range were important to preserve.

One efficient way to shift the gradient values into FP16-representable range is to scale the loss value computed in the forward pass, prior to starting back-propagation. By chain rule back-propagation ensures that all the gradient values are scaled by the same amount. This requires no extra operations during back-propagation and keeps the relevant gradient values from becoming zeros. Weight gradients must be unscaled before weight update to maintain the update magnitudes as in FP32 training. It is simplest to perform this unscaling right after the backward pass but before gradient clipping or any other gradient-related computations, ensuring that no hyper-parameters (such as gradient clipping threshold, weight decay, etc.) have to be adjusted.

There are several options to choose the loss scaling factor. The simplest one is to pick a constant scaling factor. We trained a variety of networks with scaling factors ranging from 8 to 32K (many networks did not require a scaling factor). A constant scaling factor can be chosen empir-



(a) 普通话语音识别模型的训练与验证（dev0）曲线。



(b) 普通话训练过程中的权重梯度直方图。

图2：图2a展示了三项实验的结果，分别为基准方案（FP32格式）、带有FP32主副本的伪FP16格式，以及不使用FP32主副本的伪FP16格式。图2b则显示了使用FP32格式权重进行普通话语音识别训练时权重梯度指数的直方图；在训练过程中，模型中所有层的权重梯度每隔4,000次迭代就会被采样一次。

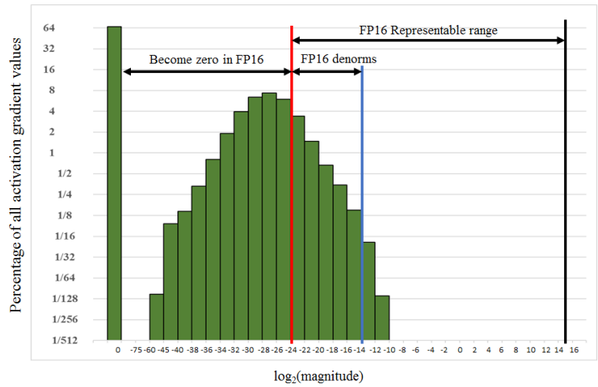


图3：Multibox SSD网络训练过程中激活梯度值的直方图。需要注意的是，x轴上的各个区间覆盖的范围各不相同，且还专门设置了用于表示零值的区间。例如，有2%的数值落在$[2^{-34}, 2^{-32})$范围内，2%的数值落在$[2^{-24}, 2^{-23})$范围内，而67%的数值为零。

幅度低于 2^{-27} 的梯度值对该模型的训练并无意义，而处于 $[2^{-27}, 2^{-24})$范围内的梯度值则对保持训练效果至关重要。

将梯度值调整到FP16格式可表示的范围的一种高效方法是在启动反向传播之前，先对前向传播中计算得到的损失值进行缩放。通过链式法则，反向传播能够确保所有梯度值都以相同的比例被缩放。这样一来，在反向传播过程中无需额外的操作，同时也能避免相关梯度值变为零。为保持与FP32训练时的更新幅度一致，权重梯度在更新之前必须先解除缩放。最简便的做法是在反向传播完成后、梯度裁剪或其他任何与梯度相关的计算之前立即执行解除缩放操作，这样就不必调整任何超参数（如梯度裁剪阈值、权重衰减等）。

选择损失缩放因子有多种方案。最简单的方法是选取一个固定的缩放因子。我们曾使用范围从8到32K不等的缩放因子训练过多种网络（其中许多网络实际上并不需要缩放因子）。固定缩放因子的选取可以通过经验方式完成，

ically or, if gradient statistics are available, directly by choosing a factor so that its product with the maximum absolute gradient value is below 65,504 (the maximum value representable in FP16). There is no downside to choosing a large scaling factor as long as it does not cause overflow during back-propagation - overflows will result in infinities and NaNs in the weight gradients which will irreversibly damage the weights after an update. Note that overflows can be efficiently detected by inspecting the computed weight gradients, for example, when weight gradient values are unscaled. One option is to skip the weight update when an overflow is detected and simply move on to the next iteration.

3.3 ARITHMETIC PRECISION

By and large neural network arithmetic falls into three categories: vector dot-products, reductions, and point-wise operations. These categories benefit from different treatment when it comes to reduced precision arithmetic. To maintain model accuracy, we found that some networks require that FP16 vector dot-product accumulates the partial products into an FP32 value, which is converted to FP16 before writing to memory. Without this accumulation in FP32, some FP16 models did not match the accuracy of the baseline models. Whereas previous GPUs supported only FP16 multiply-add operation, NVIDIA Volta GPUs introduce Tensor Cores that multiply FP16 input matrices and accumulate products into either FP16 or FP32 outputs (NVIDIA, 2017).

Large reductions (sums across elements of a vector) should be carried out in FP32. Such reductions mostly come up in batch-normalization layers when accumulating statistics and softmax layers. Both of the layer types in our implementations still read and write FP16 tensors from memory, performing the arithmetic in FP32. This did not slow down the training process since these layers are memory-bandwidth limited and not sensitive to arithmetic speed.

Point-wise operations, such as non-linearities and element-wise matrix products, are memory-bandwidth limited. Since arithmetic precision does not impact the speed of these operations, either FP16 or FP32 math can be used.

4 RESULTS

We have run experiments for a variety of deep learning tasks covering a wide range of deep learning models. We conducted the following experiments for each application:

- **Baseline (FP32)**: Single-precision storage is used for activations, weights and gradients. All arithmetic is also in FP32.
- **Mixed Precision (MP)**: FP16 is used for storage and arithmetic. Weights, activations and gradients are stored using in FP16, an FP32 master copy of weights is used for updates. Loss-scaling is used for some applications. Experiments with FP16 arithmetic used Tensor Core operations with accumulation into FP32 for convolutions, fully-connected layers, and matrix multiplies in recurrent layers.

The Baseline experiments were conducted on NVIDIA's Maxwell or Pascal GPU. Mixed Precision experiments were conducted on Volta V100 that accumulates FP16 products into FP32. The mixed precision speech recognition experiments (Section 4.3) were conducted using Maxwell GPUs using FP16 storage only. This setup allows us to emulate the TensorCore operations on non-Volta hardware. A number of networks were trained in this mode to confirm that resulting model accuracies are equivalent to MP training run on Volta V100 GPUs. This is intuitive since MP arithmetic was accumulating FP16 products into FP32 before converting the result to FP16 on a memory write.

4.1 CNNS FOR ILSVRC CLASSIFICATION

We trained several CNNs for ILSVRC classification task (Russakovsky et al., 2015) using mixed precision: Alexnet, VGG-D, GoogLeNet, Inception v2, Inception v3, and pre-activation Resnet-50. In all of these cases we were able to match the top-1 accuracy of baseline FP32 training session using identical hyper-parameters. Networks were trained using Caffe (Jia et al., 2014) framework modified to use Volta TensorOps, except for Resnet50 which used PyTorch (Paszke et al., 2017).

或者，在具备梯度统计信息的情况下，可以直接选定这样一个因子：该因子与最大绝对梯度值的乘积小于65,504（即FP16格式所能表示的最大值）。只要不会在反向传播过程中引发溢出，选用较大的缩放因子并无弊端——溢出会导致权重梯度中出现无穷大值和NaN值，进而使得权重在更新后被永久性损坏。需要注意的是，通过检查计算出的权重梯度即可高效地检测出溢出现象，比如在未对权重梯度进行缩放时。另一种处理方式是，一旦检测到溢出就跳过权重更新，直接进入下一次迭代。

3.3 算术精度

总体而言，神经网络中的算术运算可分为三类：向量点积、归约运算以及逐元素运算。在采用精度降低的算术运算时，这些不同类别需要采取不同的处理方式。为保持模型精度，我们发现某些网络要求FP16格式的向量点积将部分乘积累加为一个FP32数值，该数值在写入内存之前会先转换为FP16格式。如果没有这种FP32格式的累加机制，一些FP16模型的精度就无法达到基准方案的水平。虽然早期的GPU仅支持FP16格式的乘加运算，但NVIDIA的Volta系列GPU引入了张量核心，能够对FP16格式的输入矩阵进行乘法运算，并将乘积累加为FP16或FP32格式的输出（NVIDIA，2017年）。

向量各元素求和这类涉及大规模数值减法的操作应当使用FP32格式来执行。这类运算主要出现在需要累积统计信息的批归一化层以及softmax层中。在我们的实现中，这两种类型的层仍然会从内存中读取和写入FP16格式的张量，并以FP32格式进行算术运算。由于这些层的性能受限于内存带宽而非算术运算速度，因此并未导致训练过程变慢。

诸如非线性运算和逐元素矩阵乘法之类的点运算都受到内存带宽的限制。由于算术精度并不会影响这些运算的速度，因此可以使用FP16格式或FP32格式的数学运算。

4个结果

我们针对多种深度学习任务进行了实验，涉及范围广泛的深度学习模型。对于每项应用，我们都开展了以下实验：

- **基准方案（FP32格式）**：激活值、权重和梯度均采用单精度存储空间，所有的算术运算也以FP32格式进行。
- **混合精度（MP）**：在存储和运算过程中采用FP16格式。权重、激活值以及梯度均以FP16格式存储，而用于参数更新的权重则保留FP32格式的完整副本。某些应用场景中会使用损失缩放技术。在基于FP16格式进行运算的实验中，针对卷积操作、全连接层以及循环层中的矩阵乘法，采用了带有FP32累加功能的Tensor Core运算方式。

基准方案实验是在NVIDIA的Maxwell系列GPU或Pascal系列GPU上进行的。混合精度实验则是在Volta V100显卡上开展的，该显卡会将FP16格式的运算结果累加后再转换为FP32格式。而第4.3节中提到的混合精度语音识别实验则是仅使用Maxwell系列GPU及FP16格式的存储空间来完成的。这样的设置使得我们能够在非Volta系列的硬件上模拟TensorCore的运算功能。通过在这种模式下训练多种网络，我们验证了所得模型的准确率与在Volta V100显卡上使用混合精度训练得到的模型准确率是相当的。这一点其实很容易理解，因为混合精度运算本来就是在将FP16格式的运算结果累加为FP32格式之后，才在内存写入时将其转换回FP16格式的。

4.1 CNNSFOR竞赛ILSVRC图像识别竞赛分类任务

我们采用了混合精度训练方法，为ILSVRC分类任务（Russakovsky等人，2015年）训练了多种卷积神经网络：AlexNet模型、VGG-D模型、GoogLeNet网络、Inception v2网络、Inception v3网络以及具备预激活机制的Resnet-50网络。在所有这些案例中，即便使用完全相同的超参数，我们的模型也能达到与基准FP32训练方案相同的Top-1准确率。这些网络的训练均基于经过改进以支持Volta TensorOps技术的Caffe框架（Jia等人，2014年）进行，唯有Resnet50是使用PyTorch框架（Paszke等人，2017年）进行训练的。

Training schedules were used from public repositories, when available (training schedule for VGG-D has not been published). Top-1 accuracy on ILSVRC validation set are shown in Table 1. Baseline (FP32) accuracy in a few cases is different from published results due to single-crop testing and a simpler data augmentation. Our data augmentation in Caffe included random horizontal flipping and random cropping from 256x256 images, Resnet50 training in PyTorch used the full augmentation in the training script from PyTorch vision repository.

Table 1: ILSVRC12 classification top-1 accuracy.

Model	Baseline	Mixed Precision	Reference
AlexNet	56.77%	56.93%	(Krizhevsky et al., 2012)
VGG-D	65.40%	65.43%	(Simonyan and Zisserman, 2014)
GoogLeNet (Inception v1)	68.33%	68.43%	(Szegedy et al., 2015)
Inception v2	70.03%	70.02%	(Ioffe and Szegedy, 2015)
Inception v3	73.85%	74.13%	(Szegedy et al., 2016)
Resnet50	75.92%	76.04%	(He et al., 2016b)

Loss-scaling technique was not required for successful mixed precision training of these networks. While all tensors in the forward and backward passes were in FP16, a master copy of weights was updated in FP32 as outlined in Section 3.1.

4.2 Detection CNNs

Object detection is a regression task, where bounding box coordinate values are predicted by the network (compared to classification, where the predicted values are passed through a softmax layer to convert them to probabilities). Object detectors also have a classification component, where probabilities for an object type are predicted for each bounding box. We trained two popular detection approaches: Faster-RCNN (Ren et al., 2015) and Multibox-SSD (Liu et al., 2015a). Both detectors used VGG-16 network as the backbone. Models and training scripts were from public repositories (Girshick; Liu). Mean average precision (mAP) was computed on Pascal VOC 2007 test set. Faster-RCNN was trained on VOC 2007 training set, whereas SSD was trained on a union of VOC 2007 and 2012 data, which is the reason behind baseline mAP difference in Table 2.

Table 2: Detection network average mean precision.

Model	Baseline	MP without loss-scale	MP with loss-scale
Faster R-CNN	69.1%	68.6%	69.7%
Multibox SSD	76.9%	diverges	77.1%

As can be seen in table 2, SSD detector failed to train in FP16 without loss-scaling. By losing small gradient values to zeros, as described in Section 3.2, poor weights are learned and training diverges. As described in Section 3.2, loss-scaling factor of 8 recovers the relevant gradient values and mixed-precision training matches FP32 mAP.

4.3 Speech Recognition

We explore mixed precision training for speech data using the DeepSpeech 2 model for both English and Mandarin datasets. The model used for training on the English dataset consists of two 2D convolution layers, three recurrent layers with GRU cells, 1 row convolution layer and Connectionist temporal classification (CTC) cost layer (Graves et al., 2006). It has approximately 115 million parameters. This model is trained on our internal dataset consisting of 6000 hours of English speech. The Mandarin model has a similar architecture with a total of 215 million parameters. The Mandarin model was trained on 2600 hours of our internal training set. For these models, we run the Baseline and Pseudo FP16 experiments. All the models were trained for 20 epochs using Nesterov Stochastic Gradient Descent (SGD). All hyper-parameters such as learning rate, annealing schedule and momentum were the same for baseline and pseudo FP16 experiments. Table 3 shows the results of these experiments on independent test sets.

在可获取的情况下，训练计划参考了公共代码库中的内容（VGG-D的训练计划尚未公开）。表1展示了在ILSVRC验证集上的Top-1准确率。由于采用单张图像测试以及较为简单的数据增强方式，部分案例中的基准方案（FP32格式）准确率与已公布的数值存在差异。我们在Caffe框架中采用的数据增强方法包括随机水平翻转以及从256x256尺寸的图像中进行随机裁剪，而使用PyTorch框架训练Resnet50模型时，则采用了PyTorch视觉库训练脚本中的完整增强功能。

表1：ILSVRC12分类任务的Top-1准确率。

模型	基准方案	混合精度	参考资料
AlexNet模型	56.77%	56.93%	(Krizhevsky等人，2012年)
VGG-D模型	65.40%	65.43%	(Simonyan与Zisserman，2014年)
GoogLeNet网络（Inception v1）	68.33%	68.43%	(Szegedy等人，2015年)
Inception v2网络	70.03%	70.02%	(Ioffe与Szegedy，2015年)
Inception v3网络	73.85%	74.13%	(Szegedy等人，2016年)
Resnet50模型	75.92%	76.04%	(He等人，2016b年)

对于这些网络的混合精度训练而言，成功完成并无需使用损失缩放技术。尽管在前向传播和反向传播过程中的所有张量都采用FP16格式，但如第3.1节所述，权重的主副本仍会以FP32格式进行更新。

4.2 目标检测CNN网络

目标检测属于回归任务，网络会预测边界框的坐标值（而分类任务则是将预测值通过softmax层转换为概率）。目标检测器还包含分类组件，能够为每个边界框预测出对应物体类型的概率。我们训练了两种常用的目标检测方法：Faster-RCNN（Ren等人2015年的研究）和Multibox-SSD（Liu等人2015a年的研究）。这两种检测器均以VGG-16网络作为核心架构。相关的模型文件与训练脚本均取自公共代码库（Girshick；Liu）。平均精度均值是基于Pascal VOC 2007测试集计算得出的。Faster-RCNN在VOC 2007训练集上进行训练，而Multibox-SSD则在VOC 2007数据集与2012年数据集的合并数据集上训练，这也就是表2中基准方案mAP值存在差异的原因。

表2：检测网络的平均均值精度。

模型	基准方案	未使用损失缩放的MP模型	使用了损失缩放的MP模型
Faster R-CNN算法	69.1%	68.6%	69.7%
Multibox SSD算法	76.9%	出现偏差	77.1%

如表2所示，若不进行损失缩放，SSD检测器在FP16格式下无法完成训练。正如3.2节所描述的，当较小的梯度值被归零后，模型会学习到不良的权重，进而导致训练过程发散。按照3.2节的方法，使用值为8的损失缩放因子可以恢复这些重要的梯度值，如此一来混合精度训练得到的结果便能与FP32格式下的平均精度均值相媲美。

4.3 语音识别任务

We explore mixed precision training for speech data using the DeepSpeech 2 model for both English and Mandarin datasets. The model used for training on the English dataset consists of two 2D convolution layers, three recurrent layers with GRU cells, 1 row convolution layer and Connectionist temporal classification (CTC) cost layer (Graves et al., 2006). It has approximately 115 million parameters. This model is trained on our internal dataset consisting of 6000 hours of English speech. The Mandarin model has a similar architecture with a total of 215 million parameters. The Mandarin model was trained on 2600 hours of our internal training set. For these models, we run the Baseline and Pseudo FP16 experiments. All the models were trained for 20 epochs using Nesterov Stochastic Gradient Descent (SGD). All hyper-parameters such as learning rate, annealing schedule and momentum were the same for baseline and pseudo FP16 experiments. Table 3 shows the results of these experiments on independent test sets.

Table 3: Character Error Rate (CER) using mixed precision training for speech recognition. English results are reported on the WSJ '92 test set. Mandarin results are reported on our internal test set.

Model/Dataset	Baseline	Mixed Precision
English	2.20	1.99
Mandarin	15.82	15.01

Similar to classification and detection networks, mixed precision training works well for recurrent neural networks trained on large scale speech datasets. These speech models are the largest models trained using this technique. Also, the number of time-steps involved in training a speech model are unusually large compared to other applications using recurrent layers. As shown in table 3, Pseudo FP16 results are roughly 5 to 10% better than the baseline. This suggests that the half-precision storage format may act as a regularizer during training.

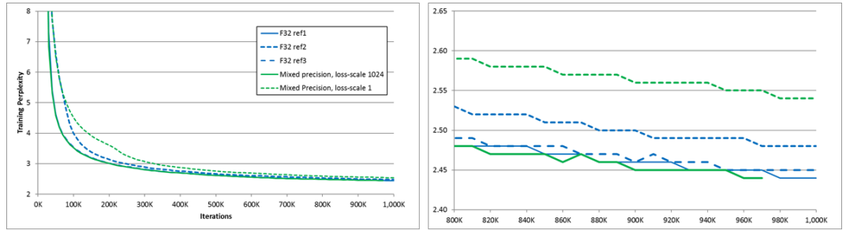


Figure 4: English to French translation network training perplexity, 3x1024 LSTM model with attention. Ref1, ref2 and ref3 represent three different FP32 training runs.

4.4 Machine Translation

For language translation we trained several variants of the model in TensorFlow tutorial for English to French translation (Google). The model used word-vocabularies, 100K and 40K entries for English and French, respectively. The networks we trained had 3 or 5 layers in the encoder and decoder, each. In both cases a layer consisted of 1024 LSTM cells. SGD optimizer was used to train on WMT15 dataset. There was a noticeable variation in accuracy of different training sessions with the same settings. For example, see the three FP32 curves in Figure 4, which shows the 3-layer model. Mixed-precision with loss-scaling matched the FP32 results, while no loss-scaling resulted in a slight degradation in the results. The 5-layer model exhibited the same training behavior.

4.5 Language Modeling

We trained English language model, designated as bigLSTM (Jozefowicz et al., 2016), on the 1 billion word dataset. The model consists of two layers of 8192 LSTM cells with projection to a 1024-dimensional embedding. This model was trained for 50 epochs using the Adagrad optimizer. The the vocabulary size is 793K words. During training, we use a sampled softmax layer with 8K negative samples. Batch size aggregated over 4 GPUs is 1024. To match FP32 perplexity training this network with FP16 requires loss-scaling, as shown in Figure 5. Without loss scaling the training perplexity curve for FP16 training diverges, compared with the FP32 training, after 300K iterations. Scaling factor of 128 recovers all the relevant gradient values and the accuracy of FP16 training matches the baseline run.

4.6 DCGAN results

Generative Adversarial Networks (GANs) combine regression and discrimination tasks during training. For image tasks, the generator network regresses pixel colors. In our case, the generator predicts three channels of 8-bit color values each. The network was trained to generate 128x128 pixel images of faces, using DCGAN methodology (Radford et al., 2015) and CelebFaces dataset (Liu et al.,

表3：采用混合精度训练处理语音识别任务时的字符错误率（CER）。英语数据基于WSJ ' 92测试集得出，而普通话数据则来自我们内部的测试集。

模型/数据集	基准方案	混合精度
英语	2.20	1.99
普通话	15.82	15.01

与分类网络和检测网络类似，混合精度训练也适用于在大规模语音数据集上训练的循环神经网络。这类语音模型正是采用该技术训练出的规模最大的模型。此外，与其他使用循环层的应用相比，训练语音模型所需的时间步数异常庞大。如表3所示，伪FP16格式的训练效果比基准方案好约5%到10%。这表明半精度存储格式可能在训练过程中起到正则化作用。

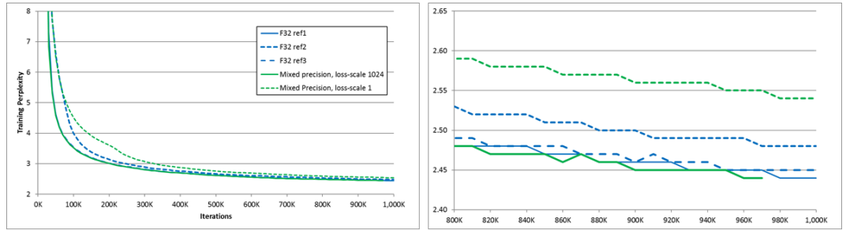


图4：英语到法语的翻译网络训练困惑度，采用带注意力机制的3x1024 LSTM模型。Ref1、ref2和ref3分别代表三次不同的FP32格式训练实验。

4.4 机器翻译

在语言翻译方面，我们基于TensorFlow教程训练了多种用于英法翻译（Google）的模型变体。这些模型采用了词表，其中英语和法语的词表规模分别为10万条和4万条。我们所训练的网络在编码器和解码器中各包含3层或5层，每层均由1024个LSTM单元构成。在WMT15数据集上训练时采用了随机梯度下降算法作为优化器。即便使用相同的设置，不同训练轮次之间的准确率也存在明显差异。例如，图4中的三条FP32格式曲线展示了3层模型的表现。采用损失缩放的混合精度训练方式与FP32格式的结果相当，而未进行损失缩放则会导致结果略有下降。5层模型也展现了类似的训练特性。

4.5 语言建模

我们在包含10亿个单词的数据集上训练了名为bigLSTM的英语语言模型（参考Jozefowicz等人2016年的研究）。该模型由两层共8192个LSTM单元构成，这些单元的输出会被映射到1024维的嵌入空间中。训练过程中采用了Adagrad优化器，共进行了50轮迭代，模型的词汇量则为79.3万词。在训练阶段，我们使用了带8千个负样本的采样softmax层。当在4块GPU上汇总批次大小时，其值为1024。为使FP16格式下的训练结果与FP32格式的困惑度指标保持一致，如图5所示，必须对损失值进行缩放处理。若不进行损失缩放，FP16格式在经过30万次迭代后，其训练困惑度曲线会与FP32格式出现显著差异。而采用128作为缩放因子后，所有相关的梯度值都能得以保留，FP16格式下的训练准确率也能与基准方案相媲美。

4.6 DCGAN模型结果

生成对抗网络（GANs网络）在训练过程中同时处理回归与判别任务。对于图像任务而言，生成器网络负责对像素颜色进行回归分析。在我们的研究中，生成器会预测三通道的8位颜色值。该网络采用DCGAN模型（Radford等人，2015年）以及CelebFaces数据集（Liu等人），经过训练后可生成128x128像素的人脸图像。

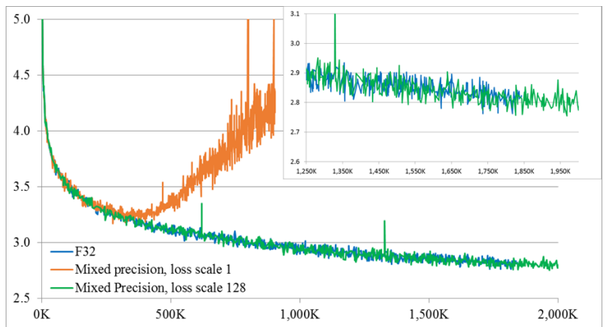


Figure 5: bigLSTM training perplexity

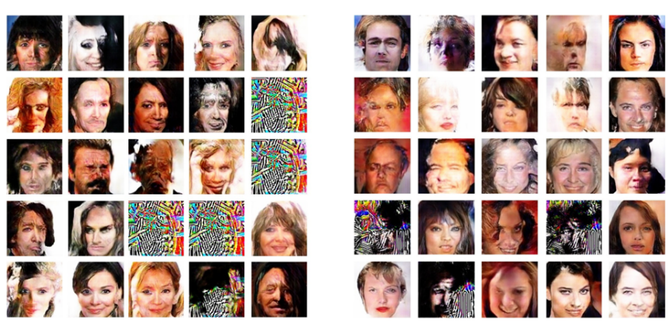


Figure 6: An uncurated set of face images generated by DCGAN. FP32 training (left) and mixed-precision training (right).

2015b). The generator had 7 layers of fractionally-strided convolutions, 6 with leaky ReLU activations, 1 with $tanh$. The discriminator had 6 convolutions, and 2 fully-connected layers. All used leaky ReLU activations except for the last layer, which used sigmoid. Batch normalization was applied to all layers except the last fully-connected layer of the discriminator. Adam optimizer was used to train for 100K iterations. An set of output images in Figure 6. Note that we show a randomly selected set of output images, whereas GAN publications typically show a curated set of outputs by excluding poor examples. Unlike other networks covered in this paper, GANs do not have a widely-accepted quantification of their result quality. Qualitatively the outputs of FP32 and mixed-precision training appear comparable. This network did not require loss-scaling to match FP32 results.

5 Conclusions and Future Work

Mixed precision training is an important technique that allows us to reduce the memory consumption as well as time spent in memory and arithmetic operations of deep neural networks. We have demonstrated that many different deep learning models can be trained using this technique with no loss in accuracy without any hyper-parameter tuning. For certain models with a large number of small gradient values, we introduce the gradient scaling method to help them converge to the same accuracy as FP32 baseline models.

DNN operations benchmarked with DeepBench[1] on Volta GPU see 2-6x speedups compared to FP32 implementations if they are limited by memory or arithmetic bandwidth. Speedups are lower when operations are latency-limited. Full network training and inference speedups depend on library

[1]https://github.com/baidu-research/DeepBench

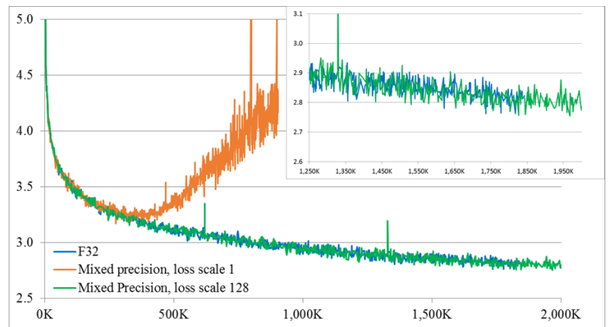


图5：bigLSTM模型的训练困惑度。

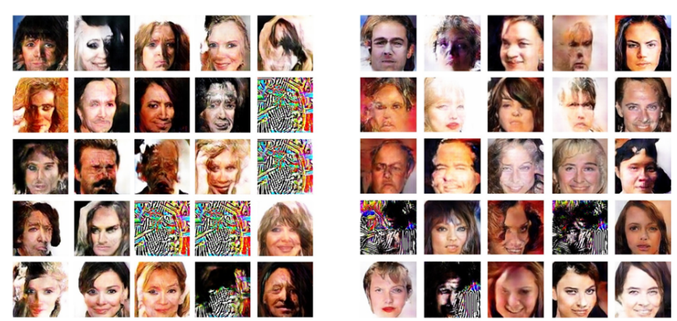


图6：由DCGAN模型生成的未经筛选的人脸图像集。左侧为FP32格式下的训练结果，右侧为混合精度训练下的结果。

2015b）。生成器包含7层步长为分数值的卷积操作，其中6层使用泄漏型ReLU激活函数，1层使用 $tanh$。鉴别器则有6层卷积操作和2层全连接层。除最后一层使用sigmoid激活函数外，所有层均采用泄漏型ReLU激活函数。除了鉴别器的最后一层全连接层外，其余所有层都应用了批量归一化处理。训练过程中采用了Adam优化器，迭代次数为10万次。图6展示了一组输出图像。需要注意的是，此处展示的是随机选取的图像集，而GAN相关研究通常会通过剔除质量较差的样本来呈现经过筛选后的输出结果。与本文讨论的其他网络不同，GANs目前尚未形成广泛认可的用于评估其结果质量的量化标准。从质量上看，FP32格式与混合精度训练得到的输出结果似乎相当。该网络无需进行损失缩放即可获得与FP32格式相近的结果。

5 CONCLUSIONS AND FUTUREWORK

混合精度训练是一项重要的技术，它能够帮助我们降低深度神经网络在内存占用以及内存操作和算术运算方面的时间消耗。我们的实验表明，多种不同的深度学习模型均可通过这种技术进行训练，且无需调整任何超参数即可保持原有的准确率。对于那些存在大量微小梯度值的模型，我们采用了梯度缩放方法，使其能够达到与FP32基准模型相同的准确度。

使用DeepBench[1] 在Volta系列GPU上对DNN运算进行的测试显示，若这些运算受到内存或算术带宽的限制，其运行速度相比FP32格式实现可提升2到6倍；而当运算受延迟限制时，提升幅度则会较小。整体网络训练与推理速度的提升程度还会取决于所使用的库以及相关的框架优化措施。

[1]https://github.com/baidu-research/DeepBench

and framework optimizations for mixed precision and are a focus of future work (experiments in this paper were carried out with early versions of both libraries and frameworks).

We would also like to extend this work to include generative models like text-to-speech systems and deep reinforcement learning applications. Furthermore, automating loss-scaling factor selection would further simplify training with mixed precision. Loss-scaling factor could be dynamically increased or decreased by inspecting the weight gradients for overflow, skipping weight updates when an overflow is detected.

此外，针对混合精度的框架优化也是未来研究的重点（本文中的实验是使用这两个库和框架的早期版本开展的）。

我们还希望将这项工作扩展到包括文本转语音系统以及深度强化学习应用等生成模型。此外，自动选择损失缩放因子将进一步简化混合精度的训练流程。通过检测权重梯度是否出现溢出情况，可以动态调整损失缩放因子，一旦检测到溢出便跳过相应的权重更新操作。

REFERENCES

D. Amodei, R. Anubhai, E. Battenberg, C. Case, J. Casper, B. Catanzaro, J. Chen, M. Chrzanowski, A. Coates, G. Diamos, et al. Deep speech 2: End-to-end speech recognition in english and mandarin. In *Proceedings of The 33rd International Conference on Machine Learning*, pages 173–182, 2016.

K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*, 2014.

M. Courbariaux, Y. Bengio, and J.-P. David. Binaryconnect: Training deep neural networks with binary weights during propagations. In C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, and R. Garnett, editors, *Advances in Neural Information Processing Systems 28*, pages 3123–3131. Curran Associates, Inc., 2015. URL http://papers.nips.cc/paper/5647-binaryconnect-training-deep-neural-networks-with-binary-weights-during-propag pdf.

R. Girshick. Faster r-cnn github repository. https://github.com/rbgirshick/py-faster-rcnn.

Google. Tensorflow tutorial: Sequence-to-sequence models. URL https://www.tensorflow.org/tutorials/seq2seq.

A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber. Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks. In *Proceedings of the 23rd international conference on Machine learning*, pages 369–376. ACM, 2006.

S. Gupta, A. Agrawal, K. Gopalakrishnan, and P. Narayanan. Deep learning with limited numerical precision. In *Proceedings of the 32nd International Conference on Machine Learning (ICML-15)*, pages 1737–1746, 2015.

A. Hannun, C. Case, J. Casper, B. Catanzaro, G. Diamos, E. Elsen, R. Prenger, S. Satheesh, S. Sengupta, A. Coates, et al. Deep speech: Scaling up end-to-end speech recognition. *arXiv preprint arXiv:1412.5567*, 2014.

K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016a.

K. He, X. Zhang, S. Ren, and J. Sun. Identity mappings in deep residual networks. In *ECCV*, 2016b.

Q. He, H. Wen, S. Zhou, Y. Wu, C. Yao, X. Zhou, and Y. Zou. Effective quantization methods for recurrent neural networks. *arXiv preprint arXiv:1611.10176*, 2016c.

S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Comput.*, 9(8):1735–1780, Nov. 1997. ISSN 0899-7667. doi: 10.1162/neco.1997.9.8.1735. URL http://dx.doi.org/10.1162/neco.1997.9.8.1735.

I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio. Binarized neural networks. In *Advances in Neural Information Processing Systems*, pages 4107–4115, 2016a.

I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio. Quantized neural networks: Training neural networks with low precision weights and activations. *arXiv preprint arXiv:1609.07061*, 2016b.

S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In F. R. Bach and D. M. Blei, editors, *ICML*, volume 37 of *JMLR Workshop and Conference Proceedings*, pages 448–456. JMLR.org, 2015. URL http://dblp.uni-trier.de/db/conf/icml/icml2015.html#IoffeS15.

Y. Jia, E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama, and T. Darrell. Caffe: Convolutional architecture for fast feature embedding. *arXiv preprint arXiv:1408.5093*, 2014.

参考文献

D. Amodei, R. Anubhai, E. Battenberg, C. Case, J. Casper, B. Catanzaro, J. Chen, M. Chrzanowski, A. Coates, G. Diamos, et al. Deep speech 2: End-to-end speech recognition in english and mandarin. In *Proceedings of The 33rd International Conference on Machine Learning*, pages 173–182, 2016.

K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, and Y. Bengio. Learning phrase representations using rnn encoder-decoder for statistical machine translation. *arXiv preprint arXiv:1406.1078*, 2014.

M. Courbariaux, Y. Bengio, and J.-P. David. Binaryconnect: Training deep neural networks with binary weights during propagations. In C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, and R. Garnett, editors, *Advances in Neural Information Processing Systems 28*, pages 3123–3131. Curran Associates, Inc., 2015. URL http://papers.nips.cc/paper/5647-binaryconnect-training-deep-neural-networks-with-binary-weights-during-propag pdf.

R. Girshick. Faster r-cnn github repository. https://github.com/rbgirshick/py-faster-rcnn.

Google. Tensorflow tutorial: Sequence-to-sequence models. URL https://www.tensorflow.org/tutorials/seq2seq.

A. Graves, S. Fernández, F. Gomez, and J. Schmidhuber. Connectionist temporal classification: labelling unsegmented sequence data with recurrent neural networks. In *Proceedings of the 23rd international conference on Machine learning*, pages 369–376. ACM, 2006.

S. Gupta, A. Agrawal, K. Gopalakrishnan, and P. Narayanan. Deep learning with limited numerical precision. In *Proceedings of the 32nd International Conference on Machine Learning (ICML-15)*, pages 1737–1746, 2015.

A. Hannun, C. Case, J. Casper, B. Catanzaro, G. Diamos, E. Elsen, R. Prenger, S. Satheesh, S. Sengupta, A. Coates, et al. Deep speech: Scaling up end-to-end speech recognition. *arXiv preprint arXiv:1412.5567*, 2014.

K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 770–778, 2016a.

K. He, X. Zhang, S. Ren, and J. Sun. Identity mappings in deep residual networks. In *ECCV*, 2016b.

Q. He, H. Wen, S. Zhou, Y. Wu, C. Yao, X. Zhou, and Y. Zou. Effective quantization methods for recurrent neural networks. *arXiv preprint arXiv:1611.10176*, 2016c.

S. Hochreiter and J. Schmidhuber. Long short-term memory. *Neural Comput.*, 9(8):1735–1780, Nov. 1997. ISSN 0899-7667. doi: 10.1162/neco.1997.9.8.1735. URL http://dx.doi.org/10.1162/neco.1997.9.8.1735.

I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio. Binarized neural networks. In *Advances in Neural Information Processing Systems*, pages 4107–4115, 2016a.

I. Hubara, M. Courbariaux, D. Soudry, R. El-Yaniv, and Y. Bengio. Quantized neural networks: Training neural networks with low precision weights and activations. *arXiv preprint arXiv:1609.07061*, 2016b.

S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In F. R. Bach and D. M. Blei, editors, *ICML*, volume 37 of *JMLR Workshop and Conference Proceedings*, pages 448–456. JMLR.org, 2015. URL http://dblp.uni-trier.de/db/conf/icml/icml2015.html#IoffeS15.

Y. Jia, E. Shelhamer, J. Donahue, S. Karayev, J. Long, R. Girshick, S. Guadarrama, and T. Darrell. Caffe: Convolutional architecture for fast feature embedding. *arXiv preprint arXiv:1408.5093*, 2014.

R. Jozefowicz, O. Vinyals, M. Schuster, N. Shazeer, and Y. Wu. Exploring the limits of language modeling, 2016. URL https://arxiv.org/pdf/1602.02410.pdf.

A. Krizhevsky, I. Sutskever, and G. E. Hinton. Imagenet classification with deep convolutional neural networks. In F. Pereira, C. J. C. Burges, L. Bottou, and K. Q. Weinberger, editors, *Advances in Neural Information Processing Systems 25*, pages 1097–1105. Curran Associates, Inc., 2012. URL http://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-networks.pdf.

W. Liu. Ssd github repository. https://github.com/weiliu89/caffe/tree/ssd.

W. Liu, D. Anguelov, D. Erhan, C. Szegedy, and S. E. Reed. Ssd: Single shot multibox detector. *CoRR*, abs/1512.02325, 2015a. URL http://dblp.uni-trier.de/db/journals/corr/corr1512.html#LiuAESR15.

Z. Liu, P. Luo, X. Wang, and X. Tang. Deep learning face attributes in the wild. In *Proceedings of International Conference on Computer Vision (ICCV)*, 2015b.

A. Mishra, E. Nurvitadhi, J. Cook, and D. Marr. Wrpn: Wide reduced-precision networks. *arXiv preprint arXiv:1709.01134, year=2017*.

NVIDIA. Nvidia tesla v100 gpu architecture. https://images.nvidia.com/content/volta-architecture/pdf/Volta-Architecture-Whitepaper-v1.0.pdf, 2017.

J. Ott, Z. Lin, Y. Zhang, S.-C. Liu, and Y. Bengio. Recurrent neural networks with limited numerical precision. *arXiv preprint arXiv:1608.06902*, 2016.

A. Paszke, S. Gross, S. Chintala, G. Chanan, E. Yang, Z. DeVito, Z. Lin, A. Desmaison, L. Antiga, and A. Lerer. Automatic differentiation in pytorch. 2017.

A. Radford, L. Metz, and S. Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. *CoRR*, abs/1511.06434, 2015. URL http://dblp.uni-trier.de/db/journals/corr/corr1511.html#RadfordMC15.

M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi. *XNOR-Net: ImageNet Classification Using Binary Convolutional Neural Networks*, pages 525–542. Springer International Publishing, Cham, 2016. ISBN 978-3-319-46493-0. doi: 10.1007/978-3-319-46493-0_32. URL https://doi.org/10.1007/978-3-319-46493-0_32.

S. Ren, K. He, R. Girshick, and J. Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *Neural Information Processing Systems (NIPS)*, 2015.

O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei. ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision (IJCV)*, 115(3):211–252, 2015. doi: 10.1007/s11263-015-0816-y.

K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.

C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. Going deeper with convolutions. In *Computer Vision and Pattern Recognition (CVPR)*, 2015. URL http://arxiv.org/abs/1409.4842.

C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna. Rethinking the inception architecture for computer vision. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2016.

Y. Wu, M. Schuster, Z. Chen, Q. V. Le, M. Norouzi, W. Macherey, M. Krikun, Y. Cao, Q. Gao, K. Macherey, et al. Google's neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.

R. Jozefowicz, O. Vinyals, M. Schuster, N. Shazeer, and Y. Wu. Exploring the limits of language modeling, 2016. URL https://arxiv.org/pdf/1602.02410.pdf.

A. Krizhevsky, I. Sutskever, and G. E. Hinton. Imagenet classification with deep convolutional neural networks. In F. Pereira, C. J. C. Burges, L. Bottou, and K. Q. Weinberger, editors, *Advances in Neural Information Processing Systems 25*, pages 1097–1105. Curran Associates, Inc., 2012. URL http://papers.nips.cc/paper/4824-imagenet-classification-with-deep-convolutional-neural-netw作品。pdf.

W. Liu. Ssd github repository. https://github.com/weiliu89/caffe/tree/ssd.

W. Liu, D. Anguelov, D. Erhan, C. Szegedy, and S. E. Reed. Ssd: Single shot multibox detector. *CoRR*, abs/1512.02325, 2015a. URL http://dblp.uni-trier.de/db/journals/corr/corr1512.html#LiuAESR15.

Z. Liu, P. Luo, X. Wang, and X. Tang. Deep learning face attributes in the wild. In *Proceedings of International Conference on Computer Vision (ICCV)*, 2015b.

A. Mishra, E. Nurvitadhi, J. Cook, and D. Marr. Wrpn: Wide reduced-precision networks. *arXiv preprint arXiv:1709.01134, year=2017*.

NVIDIA. Nvidia tesla v100 gpu architecture. https://images.nvidia.com/content/volta-architecture/pdf/Volta-Architecture-Whitepaper-v1.0.pdf, 2017.

J. Ott, Z. Lin, Y. Zhang, S.-C. Liu, and Y. Bengio. Recurrent neural networks with limited numerical precision. *arXiv preprint arXiv:1608.06902*, 2016.

A. Paszke, S. Gross, S. Chintala, G. Chanan, E. Yang, Z. DeVito, Z. Lin, A. Desmaison, L. Antiga, and A. Lerer. Automatic differentiation in pytorch. 2017.

A. Radford, L. Metz, and S. Chintala. Unsupervised representation learning with deep convolutional generative adversarial networks. *CoRR*, abs/1511.06434, 2015. URL http://dblp.uni-trier.de/db/journals/corr/corr1511.html#RadfordMC15.

M. Rastegari, V. Ordonez, J. Redmon, and A. Farhadi. *XNOR-Net: ImageNet Classification Using Binary Convolutional Neural Networks*, pages 525–542. Springer International Publishing, Cham, 2016. ISBN 978-3-319-46493-0. doi: 10.1007/978-3-319-46493-0_32. URL https://doi.org/10.1007/978-3-319-46493-0_32.

S. Ren, K. He, R. Girshick, and J. Sun. Faster R-CNN: Towards real-time object detection with region proposal networks. In *Neural Information Processing Systems (NIPS)*, 2015.

O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A. C. Berg, and L. Fei-Fei. ImageNet Large Scale Visual Recognition Challenge. *International Journal of Computer Vision (IJCV)*, 115(3):211–252, 2015. doi: 10.1007/s11263-015-0816-y.

K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556*, 2014.

C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, and A. Rabinovich. Going deeper with convolutions. In *Computer Vision and Pattern Recognition (CVPR)*, 2015. URL http://arxiv.org/abs/1409.4842.

C. Szegedy, V. Vanhoucke, S. Ioffe, J. Shlens, and Z. Wojna. Rethinking the inception architecture for computer vision. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, June 2016.

Y. Wu, M. Schuster, Z. Chen, Q. V. Le, M. Norouzi, W. Macherey, M. Krikun, Y. Cao, Q. Gao, K. Macherey, et al. Google's neural machine translation system: Bridging the gap between human and machine translation. *arXiv preprint arXiv:1609.08144*, 2016.

S. Zhou, Z. Ni, X. Zhou, H. Wen, Y. Wu, and Y. Zou. Dorefa-net: Training low bitwidth convolutional neural networks with low bitwidth gradients. *CoRR*, abs/1606.06160, 2016. URL http://arxiv.org/abs/1606.06160.

周S、倪Z、周X、文H、吴Y与邹Y。Dorefa-net：利用低位宽梯度训练低位宽卷积神经网络。*CoRR*，abs/1606.06160，2016年。网址：http://arxiv.org/abs/1606.06160。